

Tema 4

Estructuras de control



Índice

- Introducción
- Sentencias condicionales
 - *if-else*
 - *switch*
- Sentencias de iteración
 - *while*
 - *do-while*
 - *for*
 - *foreach*



Índice

- Sentencias de salto
- *break*
- *continue*
- *return*
- *goto*



1. Introducción

- En la programación, las **estructuras de control** son sentencias que permiten dirigir o modificar el flujo de la ejecución de las instrucciones que componen programa.
- Estas estructuras constituyen los pilares de la programación estructurada y nos permiten decidir qué acciones ejecutar y cuándo hacerlo, controlando así el flujo del programa de manera lógica.
- El uso correcto de estas estructuras de control es fundamental ya que será el que nos permita dotar del comportamiento deseado a nuestros programas.

1. Introducción

- Todas las estructuras de control tienen un único punto de entrada y a partir de ahí pueden modificar el **flujo de control** en base a las condiciones que se especifiquen.
- Estas condiciones de control pueden utilizar los **operadores** matemáticos, booleanos y lógicos que vimos en el tema anterior.

1. Introducción

- En este tema vamos a ver 3 tipos de estructuras de control:
- Sentencias de condición
- Sentencias de iteración
- Sentencias de salto

- Las sentencias condicionales nos permiten ejecutar diferentes bloques de código en base a una condición.
- Estas sentencias comprueban si se cumple una condición especificada y en función de ello, permiten elegir el bloque de código que se va a ejecutar a continuación.

2. Sentencias condicionales

- La sentencia `if` es una sentencia condicional que ejecuta un bloque de código únicamente si se cumple la **condición booleana** especificada entre paréntesis.
- Si se cumple la condición, se ejecutarán las instrucciones que están dentro del bloque perteneciente al `if` (delimitado por `{}`). Si no se cumple, no se ejecutan esas instrucciones y el programa continúa por la siguiente sentencia.

3. if-else

```
1 int number = 4;  
2 if(number % 2 == 0)  
3 {  
4     Console.WriteLine("El número es  
5 }
```

- Esta sentencia se puede combinar con la sentencia `else`, que ejecutará un bloque de código si no se cumplen las condiciones del `if`:

- **Únicamente** se ejecutan las instrucciones de uno de los 2 bloques. Posteriormente el programa continua su ejecución por la siguiente sentencia.

3. if-else

```
1 if(number % 2 == 0)
2 {
3     Console.WriteLine("El número es par")
```

```
4 }  
5 else  
6 {  
7     Console.WriteLine("El número es impar");  
8 }
```

- También se pueden añadir sentencias `else if` para añadir más condicionantes a la estructura de control.
- Si se cumple la primera condición, ejecuta el bloque de código y finaliza la ejecución de la estructura. Si no se cumple pregunta por la segunda condición y repite el proceso. Se pueden añadir tantos `else if` como se desee.

3. if-else

```
1  if (edad > 65)
2  {
3      Console.WriteLine("Jubilado");
```

```
4    }  
5    else if (edad > 18)  
6    {  
7        Console.WriteLine("Mayor de eda  
8    }  
9    else  
10   {
```

- Además, las estructuras `if-else` también se pueden anidar. Esto quiere decir que puede haber otras estructuras `if-else` dentro de los bloques de código a ejecutar.
- Por cada estructura `if-else` se ejecuta únicamente un bloque de código independientemente del número de `else-if` o de anidaciones que haya.

3. if-else

3. if-else

```
1  if (esMamifero == true)
2  {
3      if (esTerrestre == true)
4      {
5          Console.WriteLine("Elefante");
6      }
7      else
8      {
9          Console.WriteLine("Ballena");
10     }
11 }
12 else
13 {
14     if (vuela == true)
15     {
16         Console.WriteLine("Cigüeña");
17     }
18     else
19     {
20         Console.WriteLine("Serpiente");
21     }
22 }
```

Ejemplo de anidación:

3. if-else

Vídeo práctico

3. if-else

Vídeo práctico - código

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int number =12;
8
9             if (number > 2)
10            {
11                if (number > 10)
12                {
13                    Console.WriteLine("El número es mayor qu
14                }
15                else
16                {
17                    Console.WriteLine("El número es mayor qu
18                }
19            }
20            else if (number < -5)
21            {
```

4. switch

- La sentencia `switch` es una estructura que evalúa una expresión y ejecuta diferentes bloques de código según el valor que esa expresión tome.
- En el caso del `switch`, la expresión no está limitada a condiciones booleanas, sino que puede evaluar una variable o una sentencia.
- Es equivalente a montar una estructura `if/else-if/else` en la que se contemplen todos los casos, pero su código resulta más claro y organizado.

4. switch

Sintaxis básica:

```
1  switch (expresión)
2  {
3      case valor1:
4          // Código a ejecutar si expresión
5          break;
6      case valor2:
7          // Código a ejecutar si expresión
8          break;
9      case valor3:
10         // Código a ejecutar si expresión
11         break;
12     default:
13         // Código a ejecutar si ningún val
14         break;
15 }
```

4. switch

Sintaxis básica:

- **switch (expresión):** Evalúa la expresión o variable cuyo valor determina qué bloque de código se ejecutará.
- **case valor::** Marca un valor que la expresión puede tomar. Si la expresión coincide con ese valor, el bloque de código dentro de este case se ejecuta.

4. switch

Sintaxis básica:

- **break**:: Después de ejecutar un bloque de código asociado a un case, la sentencia break detiene la ejecución del switch y el programa continúa con la siguiente instrucción fuera del switch. Si no se coloca un break, la ejecución continúa hacia el siguiente case.
- **default**:: Este bloque se ejecuta si ningún case coincide con el valor de la expresión. Es opcional, pero recomendable para manejar valores inesperados.

4. switch

Ejemplo

```
1  int weekDay = 4;
2  switch (weekDay)
3  {
4      case 1:
5          Console.WriteLine("Lunes");
6          break;
7      case 2:
8          Console.WriteLine("Martes");
9          break;
10     case 3:
11         Console.WriteLine("Miércoles");
12         break;
13     case 4:
14         Console.WriteLine("Jueves");
15         break;
16     case 5:
17         Console.WriteLine("Viernes");
18         break;
19     case 6:
20         Console.WriteLine("Sábado");
21         break;
22     case 7:
23         Console.WriteLine("Domingo");
24         break;
25     default:
26         Console.WriteLine("Día no válido")
```

27

break:

4. switch

Ejemplo: el bloque anterior sería el equivalente al siguiente bloque combinando `if-elses`:

```
1  int weekDay = 4;
2  if (weekDay == 1)
3  {
4      Console.WriteLine("Lunes");
5  }
6  else if (weekDay == 2)
7  {
8      Console.WriteLine("Martes");
9  }
10 else if (weekDay == 7)
11 {
12     Console.WriteLine("Domingo");
13 }
14 else
15 {
16     Console.WriteLine("Día no vál
17 }
18 }
```

4. switch

- Dentro de los *cases* de un `switch` podemos utilizar operadores para comprobar las condiciones.

```
1  switch (month)
2  {
3      case < 4:
4          Console.WriteLine("Invierno");
5          break;
6      case < 7:
7          Console.WriteLine("Primavera");
8          break;
9      case < 10:
10         Console.WriteLine("Verano");
11         break;
12     case <= 12:
13         Console.WriteLine("Otoño");
14         break;
15     default:
16         Console.WriteLine("Mes no reconocido");
17         break;
18 }
```

4. switch

- También podemos combinar cases que tienen el mismo comportamiento si los ponemos secuencialmente sin break:
- En este caso, los casos 1, 3 y 5 ejecutarían el mismo código y lo mismo para los casos 2, 4 y 6.

```
1  switch (number)
2  {
3      case 1:
4      case 3:
5      case 5:
```

```
6         Console.WriteLine("Im  
7         break;  
8     case 2:  
9     case 4:  
10    case 6:  
11        Console.WriteLine("Pa  
12        break;
```

4. switch

Vídeo práctico

4. switch

Vídeo práctico - código

```
1  namespace ConsoleApp1
2  {
3      internal class Program
4      {
5          static void Main(string[] args)
6          {
7              Console.WriteLine("Elige una operación: ")
8              Console.WriteLine("1. Suma");
9              Console.WriteLine("2. Resta");
10             Console.WriteLine("3. Multiplicación");
11             Console.WriteLine("4. División");
12
13             double op1 = 5.5;
14             double op2 = 3.27;
15
16             int option = int.Parse(Console.ReadLine())
17
18             switch (option)
19             {
20                 case 1:
21                     Console.WriteLine(op1 + op2);
22                     break;
```

5. Sentencias de iteración

- Las sentencias de iteración o **bucles** son sentencias que permiten ejecutar un bloque de código repetidamente.
- Los bucles ejecutan su código de manera repetitiva, hasta que se cumpla una **condición de parada**.
- En C# tenemos disponibles las sentencias `while`, `do-while`, `for` y `foreach`.

5. Sentencias de iteración

- **¡IMPORTANTE!** Todas las sentencias de iteración deben alcanzar la condición de parada.
- Esto significa que el bucle debe interactuar con la condición para que en algún momento se cumpla su parada.
- Si un bucle no alcanza la condición de parada, se quedará ejecutándose permanentemente, haciendo que el programa se bloquee y aumentando la memoria que consume.

6. while

- El bucle `while` evalúa una condición booleana y si esta es verdadera, ejecuta el código de dentro del bucle.
- Este proceso **se repite mientras la condición siga siendo verdadera**.
- Si la condición no se cumple al iniciar el bucle, el código interno no se ejecuta ninguna vez.

```
1 while (condicion)
2 {
3     // Instrucciones
4 }
```

6. while

Ejemplo

- Repetimos la ejecución del bucle mientras se cumpla la condición (en este caso 5 veces).

```
1 int apples = 5;  
2 while (apples > 0)  
3 {  
4     Console.WriteLine("Comemos una manzana. Nos queda  
5     apples--;  
6 }
```

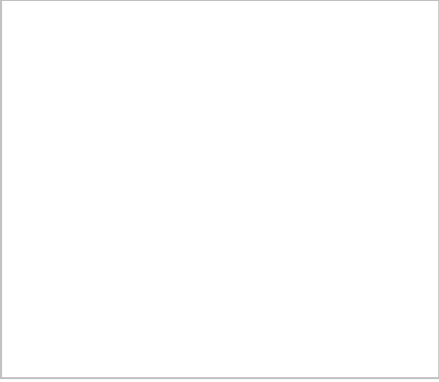


6. while

Ejemplo

- ¡Si no actuamos sobre la condición el programa se quedará en un **bucle infinito**!

```
1 int apples = 5;  
2 while (apples > 0)  
3 {  
4     Console.WriteLine("Comemos una manzana. Nos queda  
5 }
```



6. do-while

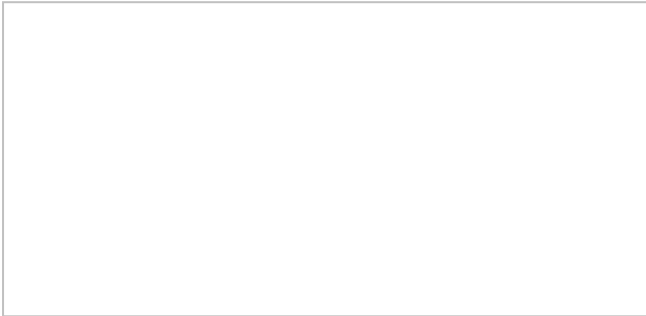
- El bucle do-while es similar al bucle while, pero con una diferencia importante: en este caso, la condición se evalúa **después** de ejecutar el bloque de código.
- Esto significa que el bloque de código dentro del bucle se ejecuta **al menos una vez**, incluso si la condición es falsa desde el principio.

```
1 do
2 {
3     // Código a ejecutar al menos una vez
4 } while (condición);
```

6. do-while

Ejemplos

```
1 int x = 0;  
2 do  
3 {  
4     Console.WriteLine("x ahora vale  
5     x++;  
6 } while (x < 10);
```



```
1 int x = 0;  
2 do  
3 {  
4     Console.WriteLine("x ahora vale  
5     x++;  
6 } while (x < 10);
```



6. do-while

Ejemplos

- En el primer ejemplo, x comienza valiendo 0, ejecuta la primera vez, pasa a valer 1 y evalúa la condición. Como es menor que 5, **sigue ejecutando** hasta cumplirla.
- En el segundo ejemplo, x comienza valiendo 0, ejecuta la primera vez, pasa a valer 1 y evalúa la condición. Como NO es mayor que 5, se cumple la **condición de parada** y el bucle termina. Pero se ha ejecutado una vez.

6. do-while

Vídeo práctico

6. do-while

Vídeo práctico - código

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int quantity = 153;
8
9             int note10 = 0;
10            int note5 = 0;
11            int coin2 = 0;
12            int coin1 = 0;
13
14            while (quantity > 0)
15            {
16                while (quantity > 10)
17                {
18                    note10++;
19                    quantity -= 10;
20                }
21                while (quantity > 5)
```

7. for

- El bucle for es una sentencia de iteración que ejecuta el código contenido un número repetido de veces, hasta que se cumple la condición de parada.
- Se trata de unos de los bucles más utilizados en la programación.
- Resulta especialmente útil cuando se sabe de antemano cuántas veces se debe repetir el bloque de código, por ejemplo, cuando recorremos un vector.

7. for

- El bucle for consta de tres partes:
- **Inicialización:** solo se ejecuta una vez. Define la variable de control del bucle.
- **Condición:** evalúa si el bucle debe continuar ejecutándose: **si es falsa, finaliza el bucle.**
- **Iterador:** se ejecuta después de cada iteración para actualizar la variable de control.

7. for

- Sintaxis:
- Dentro de la condición se puede utilizar cualquier expresión booleana, que como norma general debe estar relacionada con el iterador del bucle.

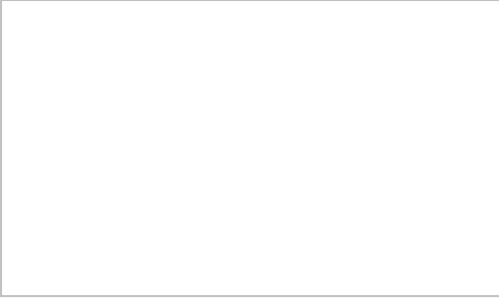
```
1 for (inicialización; condición; iterador)
2 {
3     // Código a ejecutar mientras la condición sea
```

7. for

Ejemplo

- El código se ejecuta 4 veces. En cada ejecución aumenta el valor del iterador (`i++`).

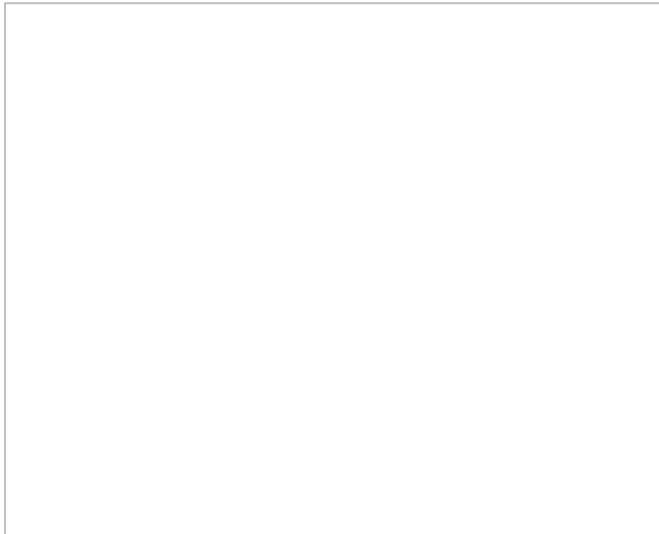
```
1 for (int i = 0; i <= 4; i++)  
2 {  
3     int number = 10;  
4     Console.WriteLine(number  
5 }
```



7. for

Ejemplo

```
1 string title = "Curso C# 13";  
2  
3 for (int i = 0; i < title.Length; i++)  
4 {  
5     Console.WriteLine("Caracter nº " + i + ": '  
6 }
```



7. for

Ejemplo

- En el ejemplo anterior utilizamos como condición que el iterador sea menor que un elemento externo al bucle (la longitud de una cadena en este caso).
- Dentro del bucle, trabajamos con la cadena utilizando el iterador como índice para recorrerla secuencialmente.

7. for

- Los bucles for, al igual que el resto de bucles, se pueden anidar.
- En ese caso, el bucle interno realiza todas sus ejecuciones cada una de las repeticiones del bucle externo.
- El for externo puede contener más código además del for interno.
- Esta estructura anidada es típica para recorrer matrices multidimensionales.

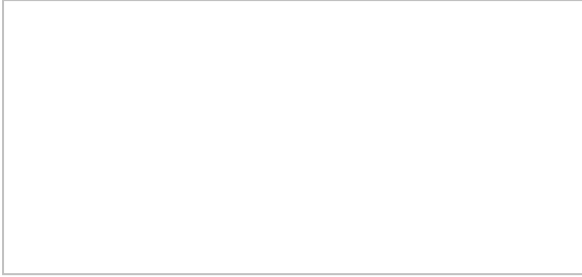
7. for

Ejemplo

- En este caso, ejecutamos 4 veces el bucle externo.
- Por cada ejecución del bucle externo, ejecutamos 10 veces el bucle interno (escribir *) y además otra instrucción (salto de línea)

```
1  int rows = 4;  
2  int columns = 10;  
3  
4  for (int i = 0; i < rows; i++)
```

```
5  {  
6    for (int j = 0; j < columns; j++)  
7    {  
8        Console.Write("*");  
9    }  
10   Console.WriteLine();  
11 }
```



8. foreach

- El bucle foreach es una sentencia de iteración que se utiliza para recorrer todos los elementos de una colección.
- A diferencia del bucle for, no hay que gestionar manualmente el índice o la condición, lo que lo hace más seguro y fácil de usar al trabajar con colecciones.

```
1 foreach (tipo itemColeccion in coleccion)
2 {
3     // Código a ejecutar para cada elemento de
```

8. foreach

Ejemplo

- En este ejemplo se recorren todos los elementos de la colección `temperatures` y se van iterando uno a uno en la variable `dailyTemperature`.

```
1 double[] temperatures = { 28.4, 27.2, 15.6, 10.2, -0.1, 12.5 };  
2 foreach (double dailyTemperature in temperatures)  
3 {
```



8. foreach

- Los bucles foreach no deben **modificar** la colección que están iterando ya que esto resultará en un error en tiempo de ejecución.
- Veremos las colecciones en detalle en el tema 6.

8. foreach

Vídeo práctico

8. foreach

Vídeo práctico - código

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             for (int i = 0; i < 5; i++)
8             {
9                 for (int j = 1; j < 8; j++)
10                 {
11                     int day = j + i * 7;
12
13                     Console.Write(day + "\t");
14                 }
15                 Console.WriteLine();
16             }
17
18             int[] ages = { 4, 18, 27, 8, 45, 89, 1, 4 };
19             int adults = 0;
20 }
```

9. Sentencias de salto

- Las **sentencias de salto** permiten alterar el flujo de ejecución de un programa de manera abrupta.
- En lugar de seguir el flujo natural (de arriba hacia abajo) o secuencial del código, estas sentencias hacen que el control del programa salte a una parte específica.
- Estas sentencias resultan útiles para controlar bucles, condicionales, o funciones.

9. Sentencias de salto

- En C# tenemos disponibles las siguientes sentencias de salto:
 - `break`
 - `continue`
 - `return`
 - `goto`

10. break

- La sentencia `break` se utiliza para salir de un bucle (`for`, `while`, `do-while`, o `foreach`).
- Cuando se encuentra una sentencia `break`, el control del programa salta fuera del bucle o de la estructura de control donde se encuentra y continúa con la siguiente instrucción después de dicha estructura.
- Ya hemos visto que se utiliza también en la sentencia `switch` para finalizar su ejecución.

10. break

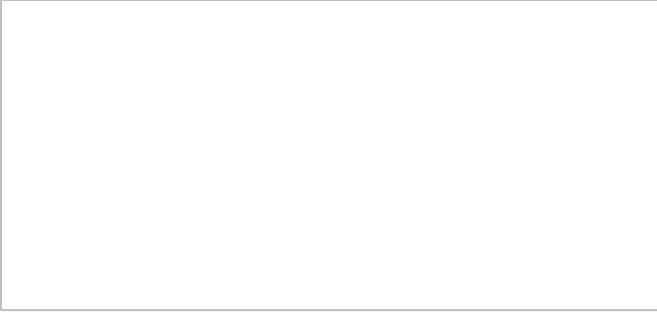
- Dentro de un bucle, break termina la ejecución de dicho bucle antes de que se cumpla la condición de salida natural.
- Es útil cuando una condición se cumple dentro del bucle y ya no tiene sentido continuar iterando.

10. break

Ejemplo

En este ejemplo, el bucle `for` se debería ejecutar 10 veces. Sin embargo, al encontrarse con la instrucción `break` cuando `i` es 5, el bucle `for` finaliza de inmediato.

```
1 for (int i = 0; i < 10; i+  
2 {  
3     if (i == 5)  
4     {  
5         break;  
6     }  
7     Console.WriteLine(i);
```



11. continue

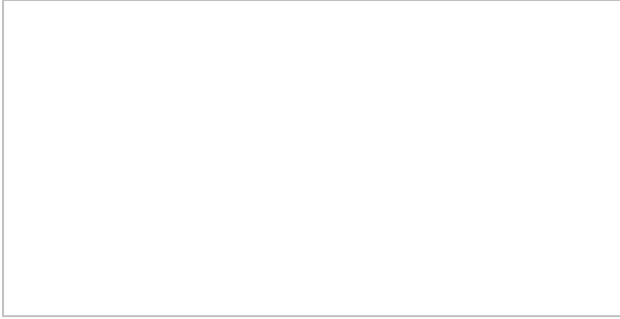
- La sentencia `continue` se utiliza dentro de bucles (`for`, `while`, `do-while`, o `foreach`) para interrumpir la **iteración actual** y pasar inmediatamente a la siguiente iteración del bucle.
- A diferencia de `break`, `continue` **no termina el bucle**, sino que salta al siguiente ciclo.

11. continue

Ejemplo

- En este ejemplo, si el número `i` es par (`i % 2 == 0`), el programa ejecuta la sentencia `continue`. De esta manera, se queda sin ejecutar el resto del bucle, por lo que los números pares se quedan sin escribir por pantalla.

```
1 for (int i = 0; i < 10; i++)
2 {
3     if (i % 2 == 0)
4     {
5         continue;
6     }
7     Console.WriteLine(i);
```



12. return

- La sentencia `return` se utiliza para salir inmediatamente de un **método**.
- Además, esta puede opcionalmente devolver un valor al lugar donde fue llamado.
- Su uso es fundamental para controlar la salida de las funciones y métodos en C#.

12. return

return sin valor

- Cuando un método **no devuelve un valor**, este es de tipo `void`.
- En ese caso, la sentencia `return` sale inmediatamente del método, continuando la ejecución por la siguiente línea a la que fue llamado.

12. return

Ejemplo

En este caso, miramos si el parámetro de entrada `name` tiene longitud 0 y si es así, hacemos un `return` para salir del método y no continuar su ejecución.

```
1 void Salute(string name)
2 {
3     if (name.Length == 0)
4     {
5         return;
6     }
7     Console.WriteLine("Hola, " + name);
}
```

12. return

return con valor

- Cuando un método **devuelve un valor**, este método es del tipo del valor devuelto (int, double, string...).
- En ese caso, la sentencia `return` devuelve un resultado al lugar donde fue llamado.

12. return

Ejemplo

En este caso, la sentencia `return` devuelve un valor `double`. Concretamente, el resultante de la operación de dividir el parámetro de entrada `a` entre 2. Este valor se recoge en la instrucción que llamó al método.

```
1 double Half(double a  
2 {  
3     return a / 2;
```

12. return

- En el siguiente tema vamos a tratar en profundidad los métodos, por lo que aprenderemos sobre su sintaxis: llamadas, parámetros, valores de retorno, etc.

13. goto

- La instrucción `goto` en C# se utiliza para transferir de manera incondicional el control de la ejecución del programa a otra parte del código.
- Aunque es una sentencia poderosa, **su uso ESTÁ DESACONSEJADO** como regla general porque puede llevar a código difícil de seguir y mantener.
- El uso de esta instrucción está ligado a menudo con la creación de código "espagueti", es decir, un flujo de control poco claro y muy difícil de seguir.

13. goto

- La instrucción `goto` se puede utilizar para:
 - Saltar a una etiqueta específica en el código.
 - Saltos en estructuras `switch`.
 - Salir de múltiples bucles anidados.

13. goto

goto con etiquetas

- Una etiqueta es un identificador seguido de dos puntos (:) que colocamos en el lugar del código donde queremos que el flujo del programa continúe.
- La instrucción `goto etiqueta;` transferirá el flujo del programa al código posterior a la etiqueta.

```
1 goto etiqueta;  
2  
3 ...  
4  
5 etiqueta:  
6 // ...
```

13. goto

Ejemplo

```
1 Console.WriteLine("Inicio");
2
3 goto controlPoint1;
4
5 controlPoint2:
6 Console.WriteLine("Punto de control 2")
7
8 goto controlPoint3;
9
10 controlPoint1:
11 Console.WriteLine("Punto de control 1")
12
13 goto controlPoint2;
14
15 controlPoint3:
16 Console.WriteLine("Punto de control 3")
17
18 Console.WriteLine("Final");
```

13. goto

- El ejemplo anterior da como resultado:
- Podemos ver en el código como seguir el flujo del programa se vuelve **muy complicado**. Es por ello que su uso no está recomendado.



13. goto

goto en switch-case

- El uso de goto dentro de una instrucción switch es poco frecuente, pero puede ser útil si deseas saltar directamente entre los diferentes casos (case).
- Utilizamos goto case en vez de break para realizar las acciones de otro case distinto al finalizar las del actual.

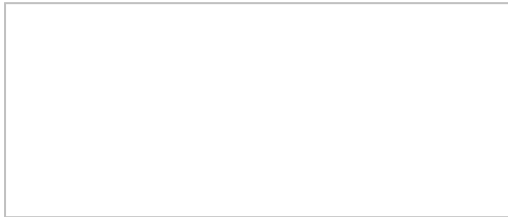
13. goto

Ejemplo

- En este ejemplo, el día 7 ejecuta su código y después ejecuta también el código del case 6.

```
1  int day = 7;  
2  
3  switch (day)  
4  {
```

```
5     case < 5:  
6         Console.WriteLine("Día labora  
7         break;  
8     case 6:  
9         Console.WriteLine("Fin de ser  
10        break;  
11    case 7:  
12        Console.WriteLine("Festivo");  
13        goto case 6;  
14    default:  
15        Console.WriteLine("Día no vá  
16        break;
```



13. goto

goto para salir de bucles anidados

- También podemos utilizar goto para salir de múltiples bucles anidados.
- Sin goto, tendríamos que utilizar varias instrucciones break o condiciones adicionales para salir de ambos bucles.

13. goto

Ejemplo

- En este ejemplo, utilizamos `goto` para salir de todos los bucles anidados cuando se cumple la condición que queremos.

```
1  for (int i = 0; i < 5; i++)  
2  {  
3      for (int j = 0; j < 5; j++)  
4      {  
5          Console.WriteLine("i: " + i + ", j: " + j);  
          if (i == 3 & j == 3) goto Exit;  
      }  
  }  
Exit;
```

```
6         if (i == 2 && j == 3)
7         {
8             Console.WriteLine("Elemento er
9             goto exit;
10        }
11    }
12 }
13
14 exit;
```



13. goto

Vídeo práctico

13. goto

Vídeo práctico - código

```
1  namespace ConsoleApp1
2  {
3      internal class Program
4      {
5          static void Main(string[] args)
6          {
7              for (int i = 0; i < 15; i++)
8              {
9                  for (int j = 1; j < 8; j++)
10                 {
11                     if (j > 31)
12                     {
13                         goto programEnd;
14                     }
15
16                     Console.Write(day + "\t");
17                 }
18                 Console.WriteLine();
19             }
20
21             programEnd:
22             Console.WriteLine("fin del bucle");
23         }
24     }
25 }
```

4. Estructuras de control

En este tema hemos aprendido las principales estructuras de control, que serán los pilares básicos para dar el comportamiento deseado a nuestros programas.

Speaker notes